

Принципы Linked Data part1

Apanovich Z.V.

IIS SBRAS, NSU

Linked Data principles

Термин *Linked Data* означает лучшие практики для публикации и связывания структурированных данных в Интернете.

Эти лучшие практики были введены Tim Berners-Lee в работе *Linked Data* .

<http://www.w3.org/DesignIssues/LinkedData.html>

и стали известны как принципы *Linked Data principles*.

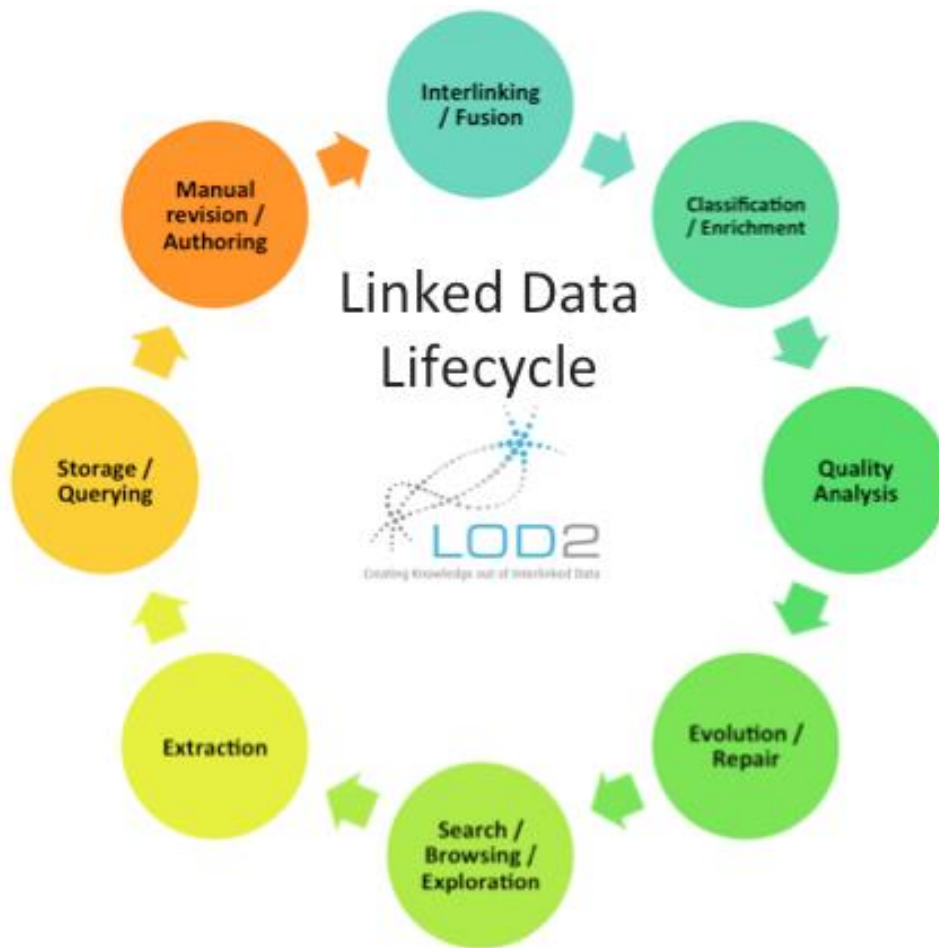


Linked Data principles

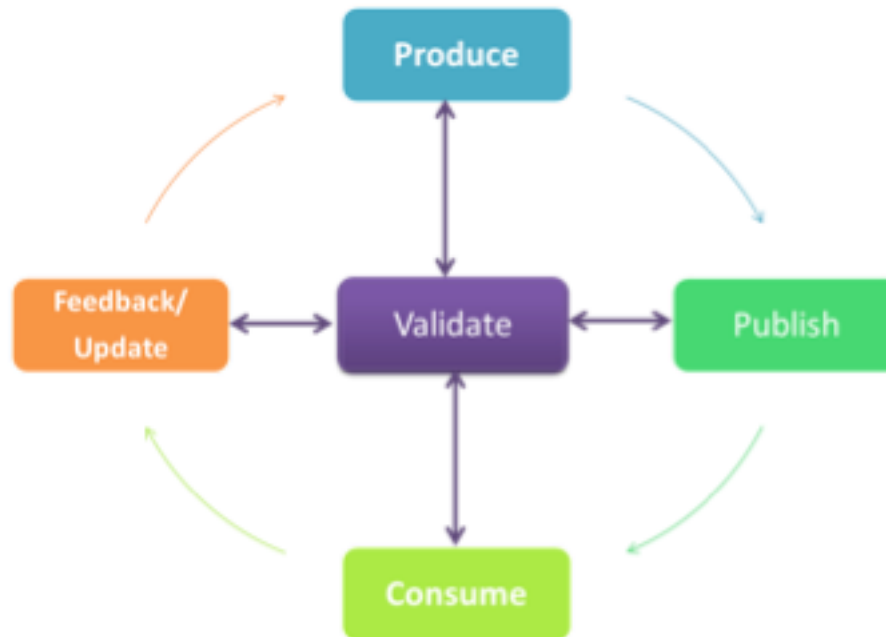
- 1) Использовать URIs в качестве имен для сущностей (**Things**)
- 2) Использовать **HTTP URIs**, так чтобы люди могли искать эти имена в Интернете.
- 3) Когда кто-то ищет URI, предоставлять полезную информацию при помощи стандартов (**RDF***, **SPARQL**).
- 4) Включать связи к другим **URIs**, так чтобы люди могли находить больше сущностей.



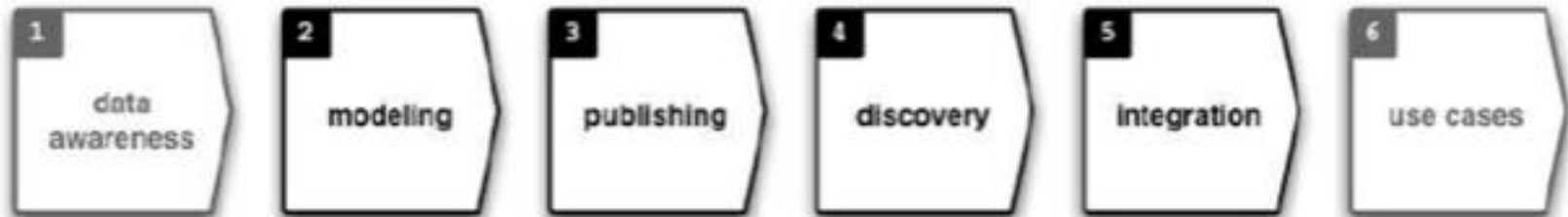
Sören Auer (2011) “The Semantic Data Web”



José M. Alvarez. (2012) “My Linked Data Lifecycle”



Michael Hausenblas (2011) “Linked Data lifecycle”



При работе с данными, встроенных в html,
меняется схема работы с данными:

- Ch Bizer «Is the Semantic Web what we expected?»
ISWC - 2016
- Обход данных (crawl)
- Нормализация словарей
- Парсинг плоских описаний
- Проверка существующих связей между данными
- Создание отсутствующих связей между данными
- Разрешение конфликтов между данными
- Соответственно, надо использовать дополнительные инструменты.

3 базовых этапа при работе со связными данными

- *Создание связанных данных:* Включает **извлечение данных, создание HTTP URI в качестве имен, и выбор словарей**. Это относится к принципам связанных данных 1 и 2, поскольку предполагает нахождение или создание HTTP URI имен для вещей.
- *Связывание данных:* Включает в себя поиск и выражение связей по наборам данных. Это относится к принципу связанных данных 4, так как это предполагает обеспечение ссылок на другие вещи.
- *Публикация связанных данных:* Включает создание метаданных о наборе данных и делание набора данных доступным для использования. Поэтому он относится к принципу связанных данных 3 в том, что он обеспечивает возвращение полезной информации о наборе данных.

Стандарты классического Web.

Базовая идея Linked Data - это применять общую архитектуру World Wide Web к задаче общего использования структурированных данных в глобальном масштабе.

The [Web of Documents](#) построен на небольшом наборе простых стандартов:

- [Uniform Resource Identifiers \(URIs\)](#) - глобальный механизм уникальных идентификаторов,
- [Hypertext Transfer Protocol \(HTTP\)](#) - универсальный механизм доступа
- [Hypertext Markup Language \(HTML\)](#) как широко используемый формат для контента
- Вдобавок, Web построен на идее создания гиперссылок между документами Web, которые могут находиться на различных серверах Web

Принцип №1 Использовать URIs в качестве имен для сущностей .

Thing <—> Неинформационный объект <->

Объект Реального Мира

URIs были первоначально определены как двух типов:

Uniform Resource Locators (**URLs**) – адреса, с конкретным расположением в сети

Uniform Resource Names (**URNs**), имена, не связанные с адресом.

В технической документации от комитетов по стандартизации, URI является предпочтительным термин и может быть использован с протоколом прикладного уровня (схема URI), таких как HTTP, чтобы быть более конкретным. Таким образом, «**HTTP URI**» является **URL**, используемый для доступа к веб-серверу.

Принцип №1 Использовать URIs в качестве имен для сущностей .

- Этот принцип предлагает раздвинуть рамки Web от обычных online ресурсов и включить **любой объект или понятие в мире**
- On the Internet, the **internationalized resource identifier (IRI)** является обобщением uniform resource identifier (URI).
- В то время как URIs ограничены подмножеством символов ASCII ,
- IRIs могут одержать символы Universal Character Set (Unicode/ISO 10646), включая Chinese or Japanese kanji, Korean, Cyrillic characters...

Принцип №2 Использовать HTTP URIs

- Протокол HTTP является универсальным механизмом доступа в Web.
- В классическом Web, HTTP URIs одновременно позволяют **globally unique identification** и простой хорошо понятный **механизм извлечения**.

Стало быть, второй принцип Linked Data предлагает использовать HTTP URIs для идентификации **объектов и абстрактных понятий**, позволяя этим URIs **dereferenced** (**разыменовываться** i.e., looked up) в соответствии с HTTP протоколом в описания идентифицированных объектов или понятий.

Разыменовывание HTTP URIs

- Разыменовывание URI - это процесс поиска URI в Интернете, чтобы получить информацию про упоминаемый ресурс.

Dereferencing HTTP URIs

В документе W3C TAG Dereferencing HTTP URIs было введено различие, как
разыменовываются URI,
идентифицирующие
информационные ресурсы (документы)
и
не-информационные ресурсы (сущности)

Информационные ресурсы:

- Когда URI, идентифицирующий **информационный ресурс** разыменовывается, сервер владельца URI обычно генерирует новое представление, текущего состояния информационного ресурса, и отправляет его обратно клиенту с помощью кода ответа **200 OK**.

Неинформационные ресурсы

Неинформационные ресурсы не могут быть разыменованы напрямую.

Веб-архитектура использует трюк, чтобы позволить URI, идентифицирующим не-информационные ресурсы, быть разыменованными:

Вместо передачи представления ресурса, сервер посылает клиенту URI информационного ресурса, который описывает не-информационный ресурс, используя код ответа HTTP 303 See Other.

Это называется 303 редирект.

На втором этапе, клиент разыменовывает этот новый URI и получает представление, описывающее исходный не-информационный ресурс

- Этот процесс называется => Content Negotiation

Принцип №3: Использовать в качестве модели данных RDF

Чтобы позволить различным приложениям обрабатывать контент Web, надо достичь соглашения относительно форматов контента.

Соглашение относительно HTML в качестве основного формата документов было важным фактором масштабирования Web.

- 3) Третий принцип Linked Data предлагает использовать единую модель для публикации структурированных данных в Web – the **Resource Description Framework (RDF)**, простую модель данных, основанную на графах, созданную для использования в контексте Web.

Принцип №4: Включать связи к другим URIs

4) Использовать гиперссылки для связи не только документов Web но и сущностей любого типа.

В отличие от классического Web, где гиперссылки не имеют типов, гиперссылки связывающие сущности в контексте Linked Data, **имеют тип**, описывающий отношения между этими сущностями.

Например,

- гиперссылка, имеющая тип *friend of* может быть создана между двумя людьми
- гиперссылка, имеющая тип *based near* может быть создана между персоной и местом

Гиперссылки в контексте Linked Data называются **RDF links** чтобы отличать их от гиперссылок между классическими документами Web .

Принцип №4: Включать связи к другим URIs

В Интернете много различных серверов отвечают за ответы на запросы пытаясь дереференцировать HTTP URIs во многих различных пространствах имен, и (в контексте Linked Data) возвращая описания RDF для ресурсов, идентифицированных этими URIs.

Поэтому в контексте Linked Data , если RDF link связывает URIs в разных пространствах имен , он связывает ресурсы разных наборов данных.

Web of Data vs Web of Documents

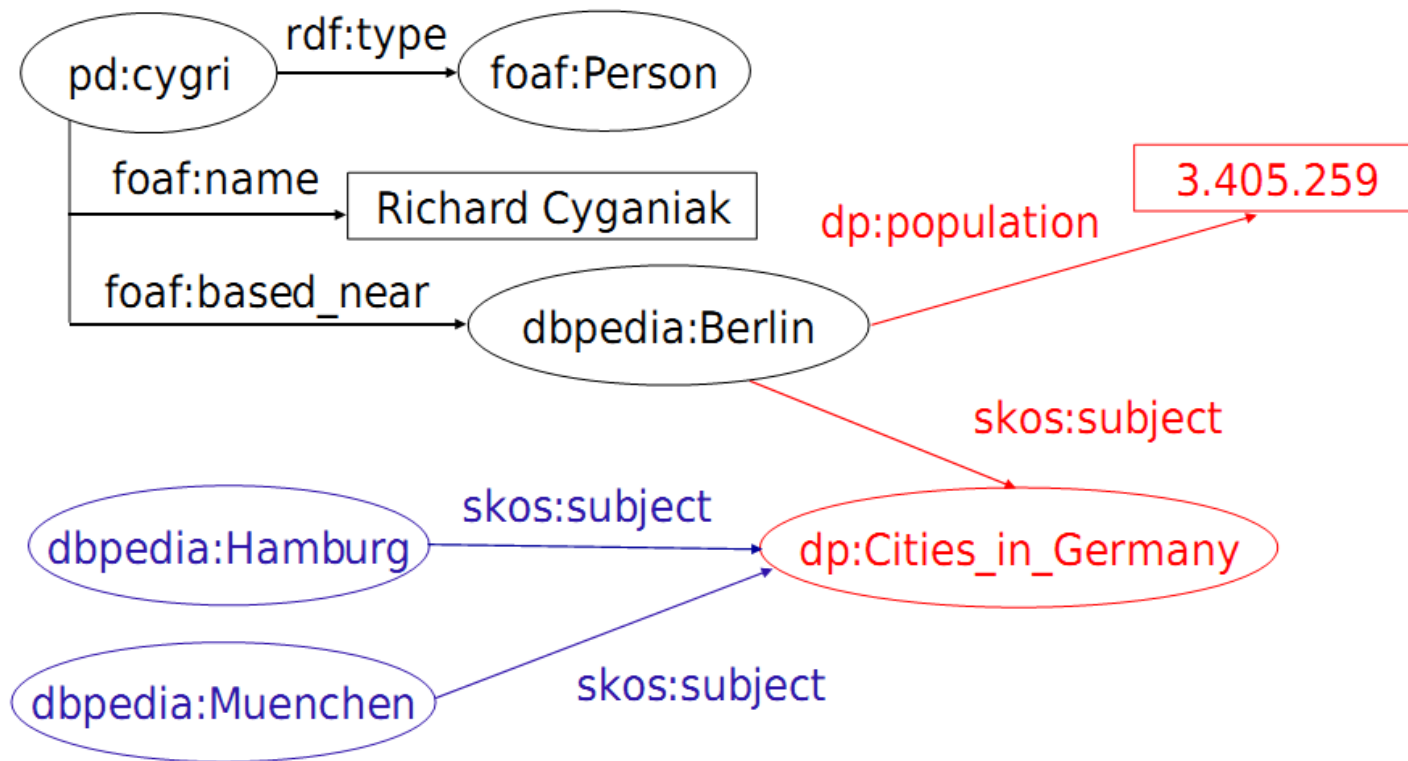
URI для документов => IRI для сущностей

HTTP => HTTP (content negotiation)

HTML => RDF

Hyperlinks => RDF links

Linked Data использует гиперсвязи для связывания распределенных данных в единое глобальное пространство данных.



Возникают новые приложения

Поскольку Web of Data основан на стандартах и общей модели данных, становится возможным реализовывать обобщенные приложения, работающие на всем пространстве данных.

- [Linked Data browsers](#)
- [Linked Data Search engines](#)
- [Linked Data mash-ups \(и более сложные приложения\)](#)

В общем, принципы Linked Data закладывают основы для расширения Web глобальным пространством данных, основанным на тех же архитектурных принципах, что и классический Web документов

Пример использования HTTP URIs



HTTP URIs

Linked Data использует только **HTTP URIs**, избегая другие схемы URI такие как **URNs** и **DOIs** .

HTTP URIs являются хорошими именами по двум причинам :

- 1) Они предоставляют простой способ создавать глобально уникальные имена децентрализованно
- 2) Они служат не только именами, но и средством доступа к информации, описывающей идентифицированную сущность.

.

URIs для ресурсов и документов

Описания ресурсов воплощены в виде веб-документов.

Описания, которые предназначены для чтения людьми, часто представлены в виде **HTML**.

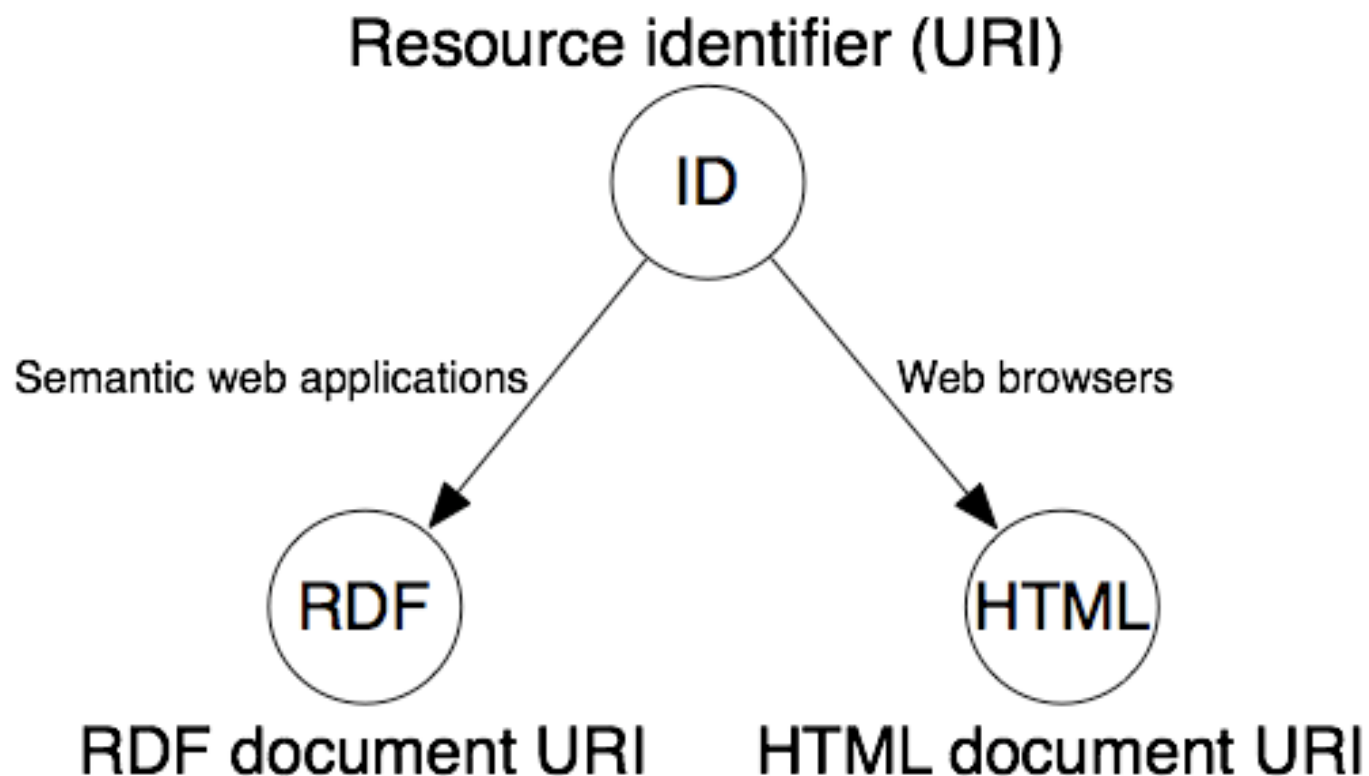
Описания, которые предназначены для потребления машинами, представлены в виде данных **RDF**.

URIs для ресурсов и документов

Там, где URI определяют объекты реального мира, важно не путать сами объекты с веб-документами, их описывающими.

Поэтому обычная практика состоит в том, чтобы использовать **различные URI**, для идентификации объекта реального мира и документа, который описывает его, чтобы избегать двусмысленностей

URIs для ресурсов и документов



Content negotiation (Обсуждение контента)

Web предназначен быть информационным пространством, которое может использоваться как людьми так и машинами.

Те и другие должны быть способны находить представления ресурсов в форме, соответствующей их потребностям,

HTML для людей,

RDF для машин.

Это может быть обеспечено при помощи механизма HTTP под названием *content negotiation* .

Content negotiation - процесс выбора наилучшего представления для запроса, если доступны несколько представлений.

303 URIs и hash URIs

Имеется две различных стратегии для того, чтобы сделать URIs, идентифицирующие **объекты реального мира**, дереференцируемыми.

Обе стратегии гарантируют что:

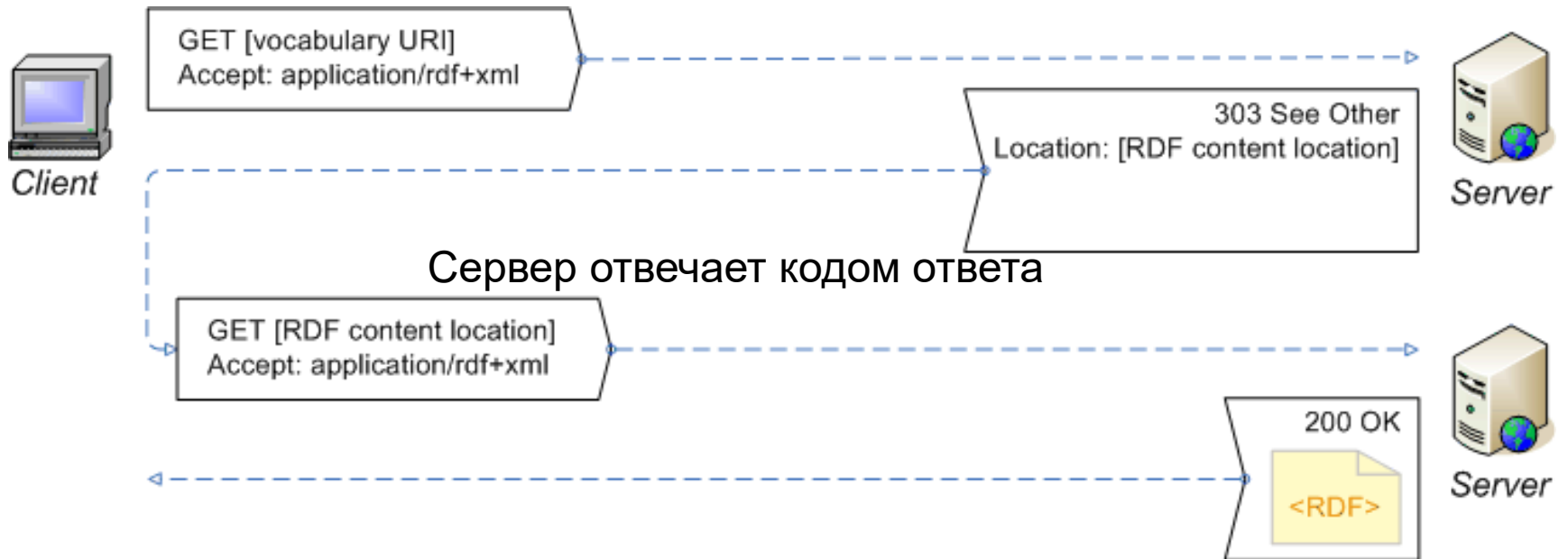
- а) объекты и документы, их описывающие, не перепутаются,
- б) и люди и машины могут найти соответствующее представление.

Эти стратегии называются **303 URIs** и **hash URIs**.

Дереференциация HTTP URI **идентифицирующего объект реального мира или абстрактное понятие** состоит из 4 шагов:

- 1) Клиент выполняет запрос HTTP GET к URI, идентифицирующему **объект реального мира или абстрактное понятие**. Если клиент- это приложение Linked Data и предпочитает представление ресурса RDF/XML, оно посылает заголовок **Accept: application/rdf+xml** вместе с запросом. Если это HTML browser, он пошлет вместо этого заголовок **Accept: text/html**.
- 2) Сервер распознает, что URI идентифицирует **объект реального мира или абстрактное понятие**. Поскольку сервер не может вернуть представление этого ресурса, он отвечает, используя код ответа **HTTP 303 See Other** и посылает клиенту URI Web документа, который описывает объект реального мира или абстрактное понятие в запрошенном формате.
- 3) Клиент выполняет запрос **HTTP GET** к URI, возвращенному сервером.
- 4) Сервер отвечает кодом ответа HTTP **200 OK** и посылает клиенту запрошенный документ, описывающий исходный ресурс в требуемом формате.

303 redirect



Пример 303 URIs

- Предположим, *Big Lynx* хочет предоставлять данные про их Managing Director Dave Smith on the Web.
- Эти данные должны быть понятны как машинам, так и людям.
- <http://biglynx.co.uk/people/dave-smith> (URI для реальной персоны Dave Smith)
- <http://biglynx.co.uk/people/dave-smith.rdf> (URI для RDF/XML документа, описывающего Dave Smith)
- <http://biglynx.co.uk/people/dave-smith.html> (URI для HTML документа, описывающего Dave Smith)

303 URIs

Step1) Чтобы получить данные RDF описывающие **Dave Smith**, клиент Linked Data свяжется с сервером <http://biglynx.co.uk/> и пошлет следующий запрос **HTTP GET** :

GET /people/dave-smith HTTP/1.1

Host: biglynx.co.uk

Accept: text/html;q=0.5, application/rdf+xml

Сервер ответит:

Step2)

- *1 HTTP/1.1 303 See Other*
2 Location: <http://biglynx.co.uk/people/dave-smith.rdf>
3 Vary: Accept

303 URIs

Step3) Затем клиент попыбует
дереференцировать URI, данный в ответе
от сервера:

1 GET /people/[dave-smith.rdf](#) HTTP/1.1

2 Host: biglynx.co.uk

3 Accept: text/html; q=0.5, application/rdf+xml

303 URIs

Step 4) Web- Сервер ответит на этот запрос, послав клиенту документ RDF/XML, содержащий данные про Dave Smith:

1 HTTP/1.1 200 OK

2 Content-Type: application/rdf+xml

3

4

5 <?xml version="1.0" encoding="UTF-8"?>

6 <rdf:RDF

7 xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"

8 xmlns:foaf="http://xmlns.com/foaf/0.1/">

9

10 <rdf:Description rdf:about="<http://biglynx.co.uk/people/dave-smith>">

11 <rdf:type rdf:resource="http://xmlns.com/foaf/0.1/Person"/>

12 <foaf:name>Dave Smith</foaf:name>

13 ...

Hash URIs, Пример

- *Big Lynx* определил различные словарные термины для описания компании в данных, публикуемых on the Web.
- Они используют стратегию hash URI для выдачи RDF/XML файла, содержащего описания этих словарных терминов.
- Поэтому *Big Lynx* присваивает URI <http://biglynx.co.uk/vocab/sme/> **rdf file!** файлу (который содержит словарь малых и средних предприятий)
- и присоединяет *fragment identifiers* к URI этого файла, чтобы идентифицировать различные словарные термины, определенные в документе.
- Таким образом, появляются следующие URIs для словарных терминов:
 - <http://biglynx.co.uk/vocab/sme#SmallMediumEnterprise>
 - <http://biglynx.co.uk/vocab/sme#Team>

Hash URIs

- Для дерекференциации этих URIs, коммуникация HTTP между клиентским приложением и сервером будет выглядеть так:
- Сначала, клиент обрезает URI, удаляя идентификатор фрагмента (e.g., #Team).
- Затем он связывается с сервером biglynx.co.uk и посылает следующий запрос HTTP GET:

1 GET /vocab/sme HTTP/1.1

2 Host: biglynx.co.uk

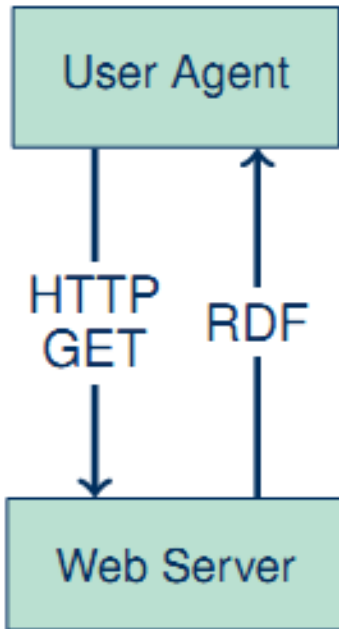
3 Accept: application/rdf+xml

Hash URIs

Сервер отвечает посылкой запрошенного документа RDF/XML, сокращенный вариант которого показан ниже:

```
1 HTTP/1.1 200 OK
2 Content-Type: application/rdf+xml;charset=utf-8
3
4
5 <?xml version="1.0"?>
6 <rdf:RDF
7   xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
8   xmlns:rdfs="http://www.w3.org/2000/01/rdf-schema#">
9
10  <rdf:Description rdf:about="http://biglynx.co.uk/vocab/sme#SmallMediumEnterprise">
11    <rdf:type rdf:resource="http://www.w3.org/2000/01/rdf-schema#Class" />
12  </rdf:Description>
13  <rdf:Description rdf:about="http://biglynx.co.uk/vocab/sme#Team">
14    <rdf:type rdf:resource="http://www.w3.org/2000/01/rdf-schema#Class" />
15  </rdf:Description>
16  ...
```

Hash URIs



<http://biglynx.co.uk/vocab/sme#Team>

<http://biglynx.co.uk/vocab/sme>

Hash versus 303

Hash URIs :

- + сокращения количества необходимых проходов HTTP (не замедляется)
- все описания, имеющие общую не-фрагментную часть URI, всегда возвращаются клиенту вместе независимо от того, нужно клиенту одно URI или все.

303 URIs:

- + Гибкость, потому что цель перенаправления может быть сконфигурирована отдельно для каждого ресурса.
- + Можно иметь по одному описывающему документу для каждого ресурса или один большой документ для всех или любую промежуточную комбинацию
- + Также возможно поменять политику позднее.

303 URIs vs Hash URIs

В результате, 303 URIs часто используются для выдачи описаний ресурсов, являющихся частью очень большого набора данных, таких как описание индивидуальных понятий из *DBpedia*, RDF-версии Wikipedia, состоящей из 3.6 миллионов понятий, которые описаны более чем 380 миллионами троек RDF.

303 URIs vs Hash URIs

- Hash URIs часто используются для идентификации терминов внутри RDF-словарей, поскольку определения RDF-словарей обычно достаточно малы (тысяча троек RDF) и часто бывает удобно для клиентского приложения получить полное определение словаря сразу, вместо того чтобы искать по одному определению.
- Hash URIs также используются, когда RDF встроен в HTML страницы при помощи RDFa

Создание HTTP URIs

На практике использование http URIs в качестве имен для сущностей, просто сводится к тому, что публикатор данных выбирает часть пространства имен, которое он контролирует, в силу обладания именем домена [http:// namespace](http://namespace) и создает URIs в этом *namespace* чтобы идентифицировать сущности в своем наборе данных.

Например *Big Lynx* владеет доменом biglynx.co.uk и его Web server запускается под <http://biglynx.co.uk/>.

- Поэтому они свободны в создании URIs в этом *namespace* и могут использовать это пространство имен для имен сущностей, про которые они хотят говорить.
- Если *Big Lynx* хочет использовать URIs для идентификации сотрудников, они могут это делать в пространстве имен <http://biglynx.co.uk/people/>.

Как создавать Cool URIs

- Основной причиной публикации Linked Data является **добавление стоимости** за счет создания входящих и исходящих ссылок.
- Поэтому, чтобы вселить уверенность в третьи стороны, желающие создавать привязки к Вашему набору данных, некоторые усилия должны быть потрачены на **создание стабильных, постоянных** идентификаторов URI для сущностей в этом наборе данных.

Простые правила при выборе URIs

- Не лезьте в пространства имен, которых вы не контролируете,
- Абстрагируйтесь от деталей реализации
- Используйте естественные ключи внутри URIs

Не лезьте в пространства имен, которых вы не контролируете !

- Когда определенный Web site рассматривается как авторитетный в определенной области, и он создает стабильные URIs для сущностей в этой области (или страницы об этих сущностях) очень заманчиво попробовать использовать (экспроприировать) эти URIs для использования в контексте Linked Data.
- Типичный пример это база данных *Internet Movie Database* <http://www.imdb.com> , которая имеет обширные данные о фильмах, актерах, продюсерах и т.д.
- Сущности описаны в документах по адресу типа: <http://www.imdb.com/title/tt0057012/>, который является URI документа о фильме *Dr. Strangelove*.
- Может показаться разумным на первый взгляд расширить этот URI идентификатором фрагмента , <http://www.imdb.com/title/tt0057012/#film>

Не лезьте в пространства имен, которых вы не контролируете !

Тем не менее, это проблематичный подход, потому что никто кроме хозяина домена imdb.com не сможет

- сделать URI dereferenceable,
- контролировать то, что будет возвращаться при дереференциации URI,
- гарантировать постоянство URI во времени.

Рекомендуемая альтернатива состоит в контексте Linked Data состоит в том, чтобы создать собственный URI для каждого фильма и устанавливать эквивалентность между этими URI и соответствующими URIs в других наборах данных везде, где это возможно возможно ([owl:sameAs](#))

Абстрагируйтесь от деталей реализации

Везде, где возможно, URI, не должны отражать **детали реализации**, которые, возможно, потребуются изменить в некоторый момент в будущем.

Например, включение в URI имени сервера или других указателей базовой технической инфраструктуры нежелательно.

Абстрагируйтесь от деталей реализации

В случае *Big Lynx*, чей сайт расположен на машине под названием **tiger** и реализован в основном на **PHP**, следующий вариант потенциального URI для RDF – документа содержащего данные про Dave Smith рассматривается как *uncool* :

- `http://tiger.biglynx.co.uk/people.php?id=dave-smith&format=rdf`

Аналогично, хозяева данных должны избегать включения имен портов, как:

- `http://tiger.biglynx.co.uk:8080/people.php?id=dave-smith&format=rdf`

Abstract away from implementation details

- В то же время, нижеприведенные URI можно рассматривать как **cool**, поскольку они менее склонны к разрушению, в случае, если сайт перенесен на другую машину, или он переделан под другой язык скриптов или программное окружение.
- <http://biglynx.co.uk/people/dave-smith.rdf>

Существуют возможности сконфигурировать Web сервер так, что детали реализации не упоминаются в URIs.

Используйте естественные ключи внутри URIs

Чтобы гарантировать уникальность URIs часто полезно основать их на некотором существующем первичном ключе, таком как уникальный ID продукта в таблице базы данных.

В случае *Big Lynx*, компания достаточно мала, чтобы комбинация имени и фамилии могла обеспечить уникальность URIs для ее сотрудников.

Это имеет достоинство создания более читаемых и запоминаемых URI.

В организациях большего размера, **employee ID number** может служить подходящей альтернативой.

Используйте естественные ключи внутри URIs

- Хороший общий принцип состоит в том, чтобы везде, где это возможно, использовать ключ, который имеет смысл в **предметной области набора данных**.
- Например, (мы будем игнорировать на данный момент вопросы, связанные с не-единственностью номера ISBN), **использование ISBN в качестве части URI** для книги предпочтительнее, чем использование ее первичного ключа из вашей внутренней базы данных.
- Это упрощает связывание данных про Вашу книгу с данными из других источников, потому что есть общий ключ, на котором могут быть основаны ссылки.

Примеры URIs

Каждая сущность, представленная в определенном наборе данных, требует создания не менее трех URIs:

- 1) URI для **объекта реального мира**
- 2) URI для соответствующего информационного объекта, описывающего объект реального мира и имеющего представление **HTML**
- 3) URI для соответствующего информационного объекта, описывающего объект реального мира и имеющего представление **RDF/XML**

Dbpedia URIs

Обычные синтаксические формы для этих URIs включают следующие примеры из DBpedia:

- http://dbpedia.org/resource/Wildlife_photography
- http://dbpedia.org/page/Wildlife_photography
- http://dbpedia.org/data/Wildlife_photography

Основным недостатком является то, что ресурс URI визуально не очень отличается от представлений RDF и HTML соответствующих описаний.

Это может быть проблематичным для новых разработчиков в области понятий Linked Data, поскольку они могут не осознавать, что URI в строке адреса навигатора изменился в связи с content negotiation и 303 [redirect](#), и безуспешно ссылаться на неправильный URI.

Альтернативной формой является:

- <http://id.biglynx.co.uk/dave-smith>
- <http://pages.biglynx.co.uk/dave-smith>
- <http://data.biglynx.co.uk/dave-smith>

Эта форма имеет плюс, что различные URIs лучше отличаются **визуально** благодаря использованию разных **подобластей**.

С точки зрения архитектуры системы это может также упростить процесс публикации Linked Data позволяя:

поставлять RDF описания ресурсов при помощи D2R Server на одной машине в подобласти **data**

как заказные скрипты на другой машине в подобласти **pages** могут выдавать навороченные HTML документы, описывающие эти ресурсы.

Скрипты в подобласти **id** будут ответственны только за исполнение content negotiation и 303 redirects.

Еще одна альтернативная форма

Третий шаблон, регулярно используемый источниками Linked Data URI, состоит в различении URIs документов от URIs объектов реального мира добавлением соответствующих расширений файлов в конце URIs.

- <http://biglynx.co.uk/people/dave-smith>
- <http://biglynx.co.uk/people/dave-smith.html>
- <http://biglynx.co.uk/people/dave-smith.rdf>

http://lod-cloud.net/versions/2014-08-30/lod-cloud_colored.svg

```
SELECT ?typedlink ?value  
WHERE {<http://dbpedia.org/resource/Novosibirsk> ?typedlink ?value  
FILTER isURI(?value)  
} LIMIT 100
```

<http://www.w3.org/2007/08/pyRdfa> - старый

<http://www.w3.org/2012/pyRdfa/> ---НОВЫЙ

Open Services Lifecycle Cr...Semantic Web Case Studi...

www.w3.org/2001/sw/sweo/public/UseCases/

СервисыBookmarksДискGoogleOn structural proper...Госдеп ответил на...Linked Data: Evolvin...Домашняя страниц...



Semantic Web Case Studies and Use Cases

Case studies include descriptions of systems that have been deployed within an organization, and are now being used within a production environment. Use cases include examples where an organization has built a prototype system, but it is not currently being used by business functions.

The list is updated regularly, as new entries are submitted to W3C. There is also an [RSS1.0 feed](#) that you can use to keep track of new submissions. Please, consult the separate [submission page](#) if you are interested in submitting a new use case or case study to be added to this list.

48entry

sorted by: [entry-type](#) and [labels](#); then by... • ☒ grouped as sorted

Case study (35)

1. [A Digital Music Archive \(DMA\) for the Norwegian National Broadcaster \(NRK\) using Semantic Web techniques](#) (Case study), by Robert Engels and Jon Roar Tonnesen, ESIS and NRK, Norway
Activity area: broadcasting
Application area of SW technologies: improved search, content discovery, and data integration
SW technologies used: RDF(S), OWL, SPARQL, and in-house vocabularies
SW technology benefits: improved search, identify new relationships, and share and re-use data

2. [A Linked Open Data Resource List Management Tool for Undergraduate Students](#) (Case study), by Chris Clarke, Talis Information Limited and University of Plymouth, United Kingdom
Activity area: education, learning technology, and publishing
Application area of SW technologies: content discovery, content management, data integration, and semantic annotation
SW technologies used: RDF, RDFa, SPARQL, RDF(S), SKOS, public datasets, and public vocabularies
SW technology benefits: explicit content relationships, personalization, reduced time to market, and share and re-use data

3. [A Semantic Web Content Repository for Clinical Research](#) (Case study), by Chimezie Ogbuji, Eugene Blackstone, and Chris Pierce, Cleveland Clinic, United States
Activity area: health care and public institution
Application area of SW technologies: data integration
SW technologies used: RDF(S), OWL, GRDDL, Rules, Rules (N3), and public vocabularies
SW technology benefits: automation, incremental modeling, and improved search

4. [An Intelligent Search Engine for Online Services for Public Administrations](#) (Case study), by Jesús Fernández Ruiz, Municipality of Zaragoza, Spain
Activity area: public institution and eGovernment
Application area of SW technologies: portal and improved search
SW technologies used: RDF(S) and in-house vocabularies
SW technology benefits: explicit content relationships, identify new relationships, and improved search

5. [An Ontology of Cantabria's Cultural Heritage](#) (Case study), by Francisca Hernández, Fundación Marcelino Botín, Spain
Activity area: public institution and museum
Application area of SW technologies: portal and data integration
SW technologies used: RDF(S) and in-house vocabularies
SW technology benefits: explicit content relationships and incremental modeling

Search facets:

Activity area

1 application lifecycle management
1 arts
3 automotive
2 broadcasting
1 cultural heritage
2 education

Application area of SW technologies

13 content discovery
6 content management
3 customization
32 data integration
4 domain modeling
23 improved search
1 lifecycle management

SW technologies used

1 eRDF
2 GRDDL
31 in-house vocabularies
23 OWL
2 OWL DL
1 OWL Lite
1 OWL-S

Benefits of SW technologies

3 automation
3 data integration
2 design online



16:27
23.09.2015